CS 4/5789 - Programming Project 3

30th September 2026

This assignment is due on **14th October 2026** at 11:59pm.

Section 0: Requirements

In this assignment, you will control a balloon navigating a 6×6 gridworld. The balloon cannot directly control its horizontal motion. Instead, it chooses whether to move up or down, while a spatially varying wind field determines its horizontal displacement.

You will solve the same navigation problem using three methods:

1. finite-horizon dynamic programming
2. infinite-horizon value iteration
3. model-free tabular Q-learning

The provided Colab notebook is the main entry point for the assignment. Use it to install dependencies, run tests, launch experiments, visualize the learned value functions and policies, and prepare the figures for your writeup. You will write the graded implementations in the provided Python files.

The assignment requires Python 3.10+ and PyTorch. To work locally, install the required packages from the assignment directory by running `pip install -r requirements.txt`. Alternatively, you may install the assignment as an editable package using `pip install -e .`. All state, value, and Q-function representations are PyTorch tensors. The provided implementation supports **CPU** device.

Complete every numbered region marked `TODO X.X`. The TODO numbers correspond to the sections in this handout. Do not change function names, argument lists, or return types used by the provided code and tests.

Section 1: Overview

The goal of this assignment is to understand the relationship between planning with a known model and learning from interaction.

In the first two parts of the assignment, the transition model and reward function are available to the agent. Dynamic programming exploits the finite horizon to solve the problem exactly by working backward in time. Value iteration instead considers an infinite-horizon discounted MDP and repeatedly applies the Bellman optimality update until convergence.

In the final part, the agent will no longer use the transition model directly. Instead, it will interact with the environment and learn action values from sampled transitions using tabular Q-learning. In particular, your implementation in `Qlearning.py` must not access `mdp.P` or `mdp.R`; it should interact with the environment only through `mdp.reset()` and `mdp.step()`.

The project contains the following files:

- `MDP.py`: Defines the balloon gridworld, including its states, actions, wind, transitions, and rewards. This file is fully implemented and should not need to be modified.
- `DP.py`: **Student code goes here.** Implements finite-horizon dynamic programming.
- `VI.py`: **Student code goes here.** Implements infinite-horizon value iteration.
- `Qlearning.py`: **Student code goes here.** Implements model-free tabular Q-learning.
- `test.py`: Contains basic sanity tests for `DP.py`, `VI.py`, and `Qlearning.py`.
- `run.py`: Provides a command-line interface for running dynamic programming, value iteration, or Q-learning individually and printing the resulting policy and value function.
- `visualize.py`: Provides visualization utilities for inspecting the value functions, Q-functions, policies, and learning behavior of dynamic programming, value iteration, and Q-learning.
- `requirements.txt`: Contains the Python package requirements for the assignment.

To run a method individually and inspect its numerical output and resulting policy, use `run.py`. Run `python run.py --help` to see the full list of available options. For example:

```
python run.py --method dp --horizon 10
python run.py --method vi --discount 0.9 --tolerance 0.01
python run.py --method qlearning --discount 0.9 --num-episodes 500
--alpha 0.5 --epsilon 0.1
```

To visualize the algorithms and their results, use `visualize.py`. The visualization utilities allow you to inspect how the value functions and policies evolve during dynamic programming and value iteration and to visualize the learning behavior of Q-learning. The specific visualization commands and experiments required for each method are described in Section 6.

We provide basic tests for the major components of dynamic programming, value iteration, and Q-learning. Run

```
python test.py
```

to execute the tests. Passing these tests does not guarantee that your implementation is correct, so you should also inspect the resulting value functions, policies, and learning curves using `run.py` and `visualize.py`.

The three algorithms solve closely related optimal-control problems, but they differ in what information is available and how that information is used:

Method	Setting	Uses transition model P?
Dynamic Programming	Finite horizon	Yes
Value Iteration	Infinite horizon	Yes
Q-learning	Infinite horizon	No

Section 2: The Balloon Navigation MDP

2.1 States

The environment is a 6×6 grid containing 36 discrete states. A state $s_{(i,j)}$ represents the current location of the balloon, where i denotes the horizontal coordinate and j denotes the vertical coordinate. The balloon begins at the initial state $s_{(1,0)}$ and must reach the blue goal state $s_{(5,5)}$. The episode terminates once the balloon reaches time step 50.

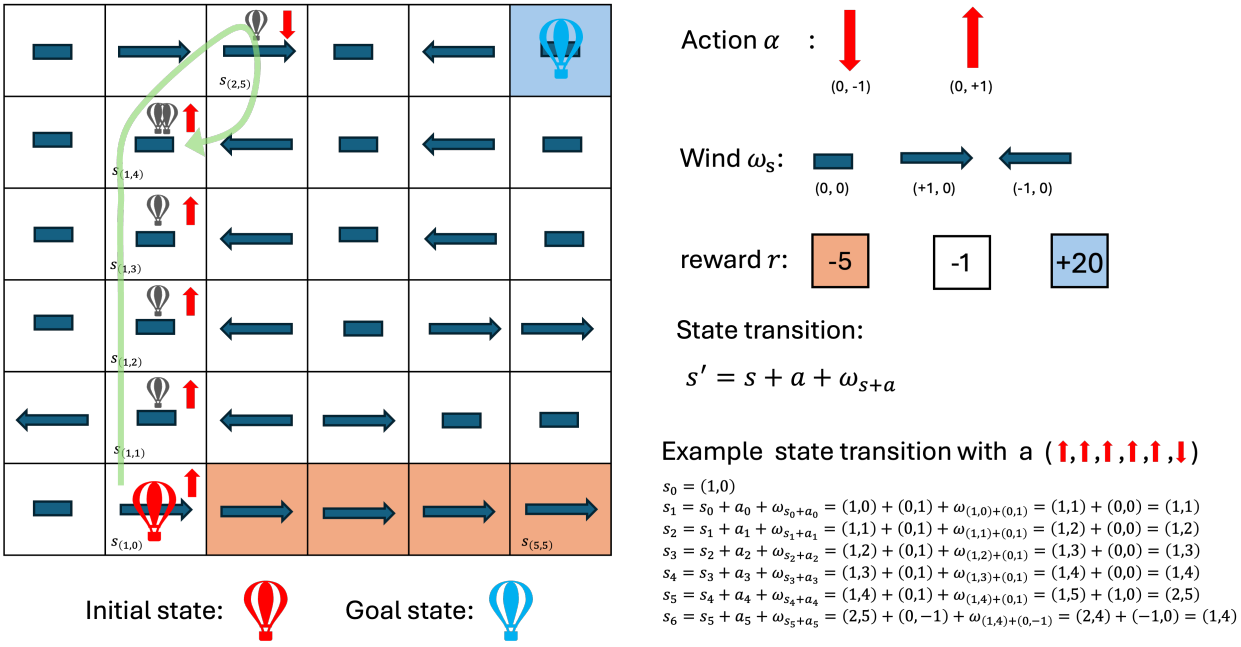


Figure 1: The balloon navigation environment. The balloon controls its vertical motion while the wind determines its horizontal displacement.

2.2 Actions

The balloon has two possible actions:

$$\mathcal{A} = \{\texttt{UP}, \texttt{DOWN}\}.$$

They correspond to the vertical displacements

$$a_{\texttt{UP}} = (0, +1), \qquad a_{\texttt{DOWN}} = (0, -1).$$

The balloon therefore has no direct horizontal control.

2.3 Wind

Each grid cell has a wind direction

$$\omega_s \in \{(-1, 0), (0, 0), (+1, 0)\},$$

corresponding respectively to leftward wind, no wind, and rightward wind. **A transition occurs in two stages**. First, the balloon attempts to execute its vertical action. The wind at the resulting cell is then applied. If the current state is s and action a is selected, let

$$\tilde{s} = s + a$$

denote the intermediate location after applying the action. The next state is then

$$s' = \tilde{s} + \omega_{\tilde{s}}.$$

The intermediate location $\tilde{s}$ is not counted as a visited state; only the final state after both the action and wind are applied is the next MDP state. At the ground and ceiling, an action that would move the balloon outside the grid is ignored. The wind is still applied. Similarly, transitions remain within the grid boundaries.

2.4 Transition Probabilities

The transition model is represented by

$$P(s'|s,a).$$

For the deterministic balloon environment, executing an action from a state produces a single next state, so

$$P(s'|s,a) = \begin{cases} 1, & \text{if } s' \text{ is the resulting state after action and wind,} \\ 0, & \text{otherwise.} \end{cases}$$

Although the environment is deterministic, we will write the algorithms using the general expectation notation so that the same Bellman equations apply to stochastic MDPs.

2.5 Rewards

The objective is to reach the goal state as quickly as possible. Each episode terminates when the balloon reaches the goal state or when the episode exceeds the maximum of 50 timesteps. The balloon receives

$$r(s,a) = \begin{cases} +20, & \text{when the transition reaches the goal,} \\ -20, & \text{when the episode terminates due to the 50-timestep limit,} \\ -5, & \text{when the transition reaches risky region,} \\ -1, & \text{otherwise.} \end{cases}$$

Once the balloon reaches the goal, the episode terminates.

2.6 Policies

We consider deterministic policies. For an infinite-horizon MDP, a policy is a mapping

$$\pi : \mathcal{S} \to \mathcal{A}.$$

We represent a deterministic policy as a Torch array of size $|\mathcal{S}|$, where the sth entry contains the action selected in state s. For finite-horizon problems, the optimal action may also depend on the remaining time. We therefore use a time-varying policy

$$\pi = (\pi_0, \pi_1, \ldots, \pi_{H-1}).$$

Section 3: Dynamic Programming: Finite-Horizon MDPs

We begin with a finite-horizon MDP. An episode contains at most H decision steps. Because the terminal time is known, we can compute the optimal value function and policy exactly by working backward from the end of the episode.

3.1 TODOs

Implement the following functions in `DP.py`:

- 3.1.1 `computeQfromV`
- 3.1.2 `dynamicProgramming`

3.2 Time-Varying Value Functions

For a finite-horizon policy

$$\pi = (\pi_0, \ldots, \pi_{H-1}),$$

the value of a state depends on the current time:

$$V_t^\pi(s) = \mathbb{E}\left[\sum_{k=t}^{H-1} r(s_k, a_k) \;\middle|\; s_t = s\right].$$

At the end of the horizon, no future reward can be collected, so

$$V_H(s) = 0 \qquad \forall s.$$

Given V_{t+1}, the action value at time t is

$$Q_t(s,a) = r(s,a) + \mathbb{E}_{s' \sim P(\cdot|s,a)}\left[V_{t+1}(s')\right].$$

Use this equation to implement `computeQfromV`.

3.3 Backward Dynamic Programming

The optimal finite-horizon policy can be computed using the following backward recursion:

- Initialize $V_H^*(s) = 0$.
- For $t = H-1, H-2, \ldots, 0$:

$$\begin{aligned} Q_t^*(s,a) &= r(s,a) + \mathbb{E}_{s' \sim P(\cdot|s,a)}[V_{t+1}^*(s')], \\ V_t^*(s) &= \max_a Q_t^*(s,a), \\ \pi_t^*(s) &= \arg\max_a Q_t^*(s,a). \end{aligned}$$

Use this recursion to implement `dynamicProgramming`. Unlike the infinite-horizon algorithms below, this procedure does not iterate until convergence. The finite horizon provides a known terminal condition, allowing the exact solution to be computed in a single backward pass.

Section 4: Value Iteration: Infinite-Horizon MDPs

We now consider an infinite-horizon discounted MDP with discount factor $0 \leq \gamma < 1$. Instead of having a fixed terminal horizon, future rewards are discounted:

$$V^{\pi}(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \;\middle|\; s_0 = s\right].$$

The optimal value function satisfies the Bellman optimality equation

$$V^*(s) = \max_a \left[r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} V^*(s')\right].$$

Value iteration finds the fixed point of this equation by repeatedly applying the Bellman optimality update.

4.1 TODOs

Implement the following functions in `VI.py`:

- 4.1.1 `computeVfromQ`
- 4.1.2 `computeQfromV`
- 4.1.3 `extractMaxPifromQ`
- 4.1.4 `valueIterationStep`
- 4.1.5 `valueIteration`

4.2 Switching Between Q and V

Given a value function V, action values can be computed using

$$Q(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V(s')].$$

Given action values, the greedy value estimate is

$$V(s) = \max_a Q(s,a).$$

Use these equations to implement `computeQfromV` and `computeVfromQ`.

4.3 Extracting a Greedy Policy

Given an action-value function Q, define its greedy policy as

$$\pi(s) = \arg\max_a Q(s,a).$$

Implement this operation in `extractMaxPifromQ`.

4.4 Value Iteration

One value-iteration update is

$$Q^{k+1}(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}\left[\max_{a'} Q^k(s',a')\right].$$

Implement this update in `valueIterationStep`. Then implement `valueIteration` by repeatedly applying the update until the change in the Q-function falls below the provided convergence tolerance. For example, convergence can be measured using

$$\epsilon_k = \|Q^{k+1} - Q^k\|_{\infty}.$$

After convergence, extract the greedy policy and corresponding value function from the final Q-function.

Section 5: Tabular Q-Learning

Dynamic programming and value iteration both use the transition model $P(s'|s,a)$ to compute expectations over possible next states. In many real problems, however, the transition model is unknown. Q-learning instead learns an action-value function directly from interaction with the environment. At each step, the agent observes a sampled transition (s_t, a_t, r_t, s_{t+1}) and uses it to update its estimate of Q. In this section, your Q-learning implementation should interact with the environment through sampled transitions. It should not use the transition matrix to perform Bellman backups.

5.1 TODOs

Implement the following functions in `Qlearning.py`:

- 5.1.1 `epsilonGreedy`
- 5.1.2 `qLearningUpdate`
- 5.1.3 `qLearning`

5.2 ϵ-Greedy Exploration

If the agent always selects its current greedy action, it may fail to discover actions whose values have not yet been accurately estimated. We therefore use an ϵ-greedy behavior policy. Given the current Q-function, the agent chooses

$$a = \begin{cases} \text{a uniformly random action}, & \text{with probability } \epsilon, \\ \arg\max_{a'} Q(s,a'), & \text{with probability } 1-\epsilon. \end{cases}$$

Implement this action-selection rule in `epsilonGreedy`.

5.3 Sample-Based Bellman Targets

Suppose the agent observes a transition (s_t, a_t, r_t, s_{t+1}). The one-step Q-learning target is

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a').$$

The corresponding temporal-difference error is

$$\delta_t = y_t - Q(s_t, a_t).$$

The tabular Q-learning update is

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\delta_t,$$

or equivalently,

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right].$$

If s_{t+1} is terminal, there is no future return and the target should contain only the observed reward. Implement this update in `qLearningUpdate`.

5.4 Learning From Interaction

Finally, implement `qLearning`. Starting from an initialized Q-table, repeatedly:

1. reset the environment,
2. select actions using the ϵ-greedy policy,
3. observe the resulting transition,
4. update the corresponding Q-value, and
5. continue until the episode terminates.

After training, return the learned Q-function and its greedy policy. The important distinction from value iteration is that Q-learning does not compute $\mathbb{E}_{s' \sim P(\cdot|s,a)}[\max_{a'} Q(s', a')]$ using the known transition model. Instead, each environment interaction provides a sample of the Bellman target.

Section 6: Testing and Visualization

We provide basic tests for each TODO function runs and returns outputs with the expected shapes/types. Passing these tests does not guarantee that your implementation is correct, so you should also inspect the resulting value functions, policies, and learning curves by yourself.
Estimated runtime on a **Colab CPU** is less than 1 minute for each experiment.

6.1 Dynamic Programming

Visualize the finite-horizon optimal value function and policy at different time steps. Run dynamic programming for horizons $\{1, 3, 5, 7, 9\}$ For each horizon, visualize $V_0^*(s)$ and $\pi_0^*(s)$. For one longer horizon, also step through the time-dependent value functions and policies $(V_0^*, \pi_0^*), (V_1^*, \pi_1^*), \ldots, (V_{H-1}^*, \pi_{H-1}^*)$. Observe how the policy changes as the remaining horizon decreases.

6.2 Value Iteration

Initialize the Q-function to zero and visualize several successive value-iteration updates. In particular, observe how information about the goal propagates through the grid as Bellman updates are repeatedly applied. After convergence, visualize the final $V^*(s)$ and $\pi^*(s)$. Repeat the experiment for multiple discount factors, such as

$$\gamma \in \{0.9, 0.99, 0.999\}.$$

Record the number of iterations required for value iteration to satisfy the same convergence tolerance for each value of γ.

6.3 Q-Learning

Train tabular Q-learning on the same infinite-horizon problem used for value iteration. Plot the learning progress as a function of environment interactions or training episodes. Compare the learned Q-function with the optimal Q-function obtained from value iteration.

Because Q-learning starts from a fixed initial state and learns only from sampled interactions, we evaluate its performance over the set of states reachable from the initial state, denoted by $\mathcal{S}_{\text{reach}}$. One useful error measure is

$$E_k = \max_{s \in \mathcal{S}_{\text{reach}},\, a \in \mathcal{A}} \left| Q_k(s,a) - Q_{\text{VI}}^*(s,a) \right|,$$

where Q_{VI}^* is the converged Q-function obtained using value iteration.

You may also compare the learned greedy policy with the value-iteration policy. Since the goal state is terminal and its greedy action is not meaningful, define

$$\mathcal{S}_{\text{eval}} = \mathcal{S}_{\text{reach}} \setminus \{s_{\text{goal}}\}.$$

The policy agreement is

$$A_k = \frac{1}{|\mathcal{S}_{\text{eval}}|} \sum_{s \in \mathcal{S}_{\text{eval}}} \mathbf{1}\left[\arg\max_a Q_k(s,a) = \pi_{\text{VI}}^*(s)\right].$$

Finally, repeat Q-learning using different exploration rates, for example

$$\epsilon \in \{0, 0.1, 0.5\},$$

while keeping the remaining training settings fixed.

Section 7: Experiments and Discussion

Include the requested figures and answer the following questions in your writeup. A few well-supported sentences are sufficient for each question.
`visualize.py` already includes all the plots and outputs necessary for the discussion questions. You may implement additional visualizations or print additional outputs if you would like to further support your explanations. Please note that the autograder assumes the default settings and code structure provided in the starter code, except for the designated TODO sections in `VI.py`, `DP.py`, and `Qlearning.py`. Therefore, make sure your implementations work correctly with the provided default settings.

1. **Finite horizon and time dependence.** Compare the optimal value function and policy at timestep $t = 0$ for different horizons H. What changes as the horizon becomes longer? What happens when the remaining horizon is too short for the balloon to reach the goal?
2. **Discounting and value propagation.** Compare value iteration for different values of γ. How does γ affect the converged value function, optimal policy, and the number of iterations required for convergence?
3. **Dynamic programming versus value iteration.** Compare the policy produced by finite-horizon dynamic programming for a sufficiently long horizon with the infinite-horizon policy produced by value iteration for a discount factor close to 1. In what ways are they similar, and why should they not necessarily be identical?
4. **Value iteration versus Q-learning.** Both algorithms ultimately attempt to find an optimal Q-function. Explain the key difference between the information used by their Bellman updates. Why can value iteration update every state-action pair directly, while Q-learning must gather experience from the environment?
5. **Exploration in Q-learning.** Compare Q-learning with different values of ϵ. How does the amount of exploration affect the rate at which the learned Q-function and greedy policy approach the value-iteration solution? What problem can occur when $\epsilon = 0$?

Section 8: Submission

Your submission should contain:

- your completed `DP.py`,
- your completed `VI.py`,
- your completed `Qlearning.py`, and
- a PDF writeup containing the requested plots and answers to Section 7.

Before submitting, run all provided tests and verify that your code runs from the provided notebook without modifying the required function interfaces. Zip the required files and submit it to Gradescope.